Technische Universität Braunschweig

A Branch-and-Bound Method for Bilevel Parallel Machine Scheduling with Adversarial Job Selection

Master Thesis Presentation

Ole Damke, April 2026
Institut für Mathematische Optimierung
Technische Universität Braunschweig

Outline

- Motivation & Problem
- Branch-and-Bound Algorithm
- The Ceiling Bound
- The Max-LPT Bound
- Early Optimality Detection
- Computational Experiments
- Conclusion

Motivation

Scenario: production facility

- Manager must decide which jobs to accept within a fixed budget
- Scheduler will assign jobs to machines to minimise completion time
- Manager needs a **worst-case guarantee**: what is the maximum possible makespan?

This is a **bilevel optimisation problem**:

- **Leader** (manager): adversarially selects jobs to maximise makespan
- **Follower** (scheduler): schedules jobs to minimise makespan

Our problem combines

Upper level (Leader)
Unbounded Knapsack

$\downarrow$ selects job portfolio

Lower level (Follower)
Parallel machine scheduling
$(P \| C_{max})$

Bilevel Optimisation Framework

Definition (Bilevel Optimisation Problem)

$$\begin{aligned} \max_{x} \quad & F(x, y^*(x)) \\ \text{s.t.} \quad & x \in X \\ \text{where} \quad & y^*(x) \in \arg\min_{y} \{f(x, y) : y \in Y(x)\} \end{aligned}$$

- The **leader** chooses x, anticipating the follower's optimal response $y^*(x)$
- The **follower** observes x and optimises their own objective
- **Optimistic formulation:** if follower has multiple optima, pick the one best for the leader

Computational difficulty

misation is Σ_2^P-hard in general. Our variant combines two NP-hard subproblems (knapsack + scheduling) with a non-convex,

The Knapsack-Scheduling Problem

Problem data: n job types, m identical parallel machines, budget $B > 0$

- Job type i: duration $d_i > 0$, cost $p_i > 0$; items sorted $d_1 \geqslant \cdots \geqslant d_n$

Leader (Upper Level)

$$\max_{x \in \mathbb{Z}_+^n} \quad C_{\max}^{\mathrm{OPT}}(x)$$
$$\text{s.t.} \quad \sum_{i=1}^{n} p_i x_i \leqslant B$$

x_i = # jobs of type i selected

Follower (Lower Level)

$$C_{\max}^{\mathrm{OPT}}(x) = \min_{y \in \mathbb{Z}_+^{n \times m}} \quad \max_k L_k$$
$$\text{s.t.} \quad \sum_k y_{ik} = x_i \quad \forall i$$

$L_k = \sum_i d_i y_{ik}$: load on machine k

Leader **maximises** $C_{\max}^{\mathrm{OPT}}(x)$; follower **minimises** it. The follower's problem is solved optimally via integer programming (Gurobi).

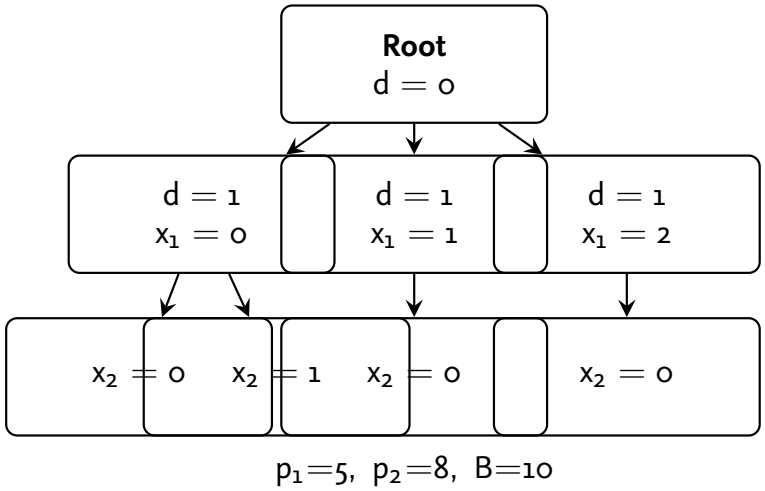
Why Enumeration Fails

Naive approach: enumerate all feasible x, solve follower for each.

Max copies of type i: $x_i \leqslant \lfloor B/p_i \rfloor$

Search space upper bound:

$$\prod_{i=1}^{n} \left(\left\lfloor \frac{B}{p_i} \right\rfloor + 1 \right)$$



Example

$n = 10, \ B = 100, \ p_i = 5 + i$

Upper bound: $\mathbf{5.29 \times 10^{10}}$ combinations

At 1 ms per solve: $\approx$ **1.68 years**

Branch-and-Bound: Overview

Core idea: explore a search tree; prune branches that cannot beat the incumbent.

Three components:

1. **Branching** — fix the next item quantity x_{d+1}
2. **Bounding** — compute an upper bound on the best makespan reachable from a node
3. **Pruning** — discard if $UB \leqslant C_{max}^{inc}$

Node at depth d

- **Fixed quantities:** $x_1, \ldots, x_d$
- **Remaining budget:** $B_{rem} = B - \sum_{i=1}^{d} p_i x_i$
- **Machine loads:** LPT schedule for items $1, \ldots, d$ — maintained incrementally
- **Branching:** branch on $x_{d+1} = 0, 1, \ldots, \lfloor B_{rem}/p_{d+1} \rfloor$

The LPT Rule as Follower's Algorithm

Longest Processing Time (LPT) rule [Graham 1969]:

- Sort jobs by non-increasing duration
- Greedily assign each job to the least-loaded machine

Key property: LPT is a $\frac{4}{3}$-approximation for $P||C_{max}$

LPT Scheduling

Input: jobs sorted desc., m machines

$L_1 = \cdots = L_m \leftarrow 0$

for each job j (desc. d_j):

$k^* \leftarrow \arg\min_k L_k$

$L_{k^*} \leftarrow L_{k^*} + d_j$

return $C_{max}^{LPT} = \max_k L_k$

Complexity: $O(J \log m)$

Often **optimal** in practice — confirmed in our experiments.

Incremental update

When branching from depth d to $d+1$:

add x_{d+1} jobs of duration d_{d+1} to the LPT state.

No full recomputation!

Pruning via Upper Bounds

At depth d with remaining budget B_{rem}:

$$\max_{\substack{x_{d+1},\ldots,x_n \geqslant 0 \\ \sum_{i>d} p_i x_i \leqslant B_{rem}}} C_{max}^{OPT}(x_1, \ldots, x_n) \leqslant UB(x_1, \ldots, x_d, B_{rem})$$

Prune whenever: $UB(x_1, \ldots, x_d, B_{rem}) \leqslant C_{max}^{inc}$

Upper bound decomposition:

$$UB = \underbrace{C_{max}^{LPT,(d)}}_{\text{fixed items}} + \underbrace{\text{best extra contribution from items } d{+}1, \ldots, n}_{\text{relaxation}}$$

We develop **two relaxations**:

- **Ceiling bound** — knapsack scheduling relaxation using $\lceil y_i/m \rceil$
- **Max-LPT bound** — maximise LPT makespan over remaining items

Ceiling Bound: Formulation

Knapsack Scheduling Relaxation

$$\mathrm{UB} = C_{\max}^{\mathrm{LPT},(d)} + \max_{y_{d+1},\ldots,y_n \geqslant 0} \sum_{i=d+1}^{n} d_i \left\lceil \frac{y_i}{m} \right\rceil \quad \text{s.t.} \quad \sum_{i=d+1}^{n} p_i y_i \leqslant B_{\mathrm{rem}}$$

Intuition: on m equally-loaded machines, adding y_i jobs of type i increases makespan by exactly $\lceil y_i/m \rceil \cdot d_i$ — an overestimate since machines are rarely perfectly balanced.

DP implementation — O(1) lookups at each node

Reduce to standard knapsack. For each type i create:

- Unit item: cost p_i, value d_i (covers 1 to $m-1$ copies)
- Bulk package: cost $m \cdot p_i$, value d_i (batch of m)

e in reverse item order $\Rightarrow$ depth-d query is a single table lookup.

Ceiling Bound: Validity Proof Sketch

Theorem (Upper Bound Validity)

For any completion $(x_{d+1}, \ldots, x_n)$: $C_{max}^{OPT}(x_1, \ldots, x_n) \leqslant UB$

Proof sketch:

1. LPT is $\frac{4}{3}$-approx., so $C_{max}^{OPT}(x) \leqslant C_{max}^{LPT}(x)$ for all x
2. After scheduling fixed items via LPT: makespan $= C_{max}^{LPT,(d)}$
3. Assume all machines filled to $C_{max}^{LPT,(d)}$ (worst-case — only inflates bound)
4. Adding y_i jobs of type i on m equally-loaded machines increases makespan by $\lceil y_i/m \rceil \cdot d_i$
5. Hence: $C_{max}^{LPT}(x) \leqslant C_{max}^{LPT,(d)} + \sum_{i>d} \lceil x_i/m \rceil d_i$
6. Maximise the RHS over the remaining budget $\Rightarrow$ bound $\square$

The Max-LPT Problem

Definition (Max-LPT Problem)

Find $x \in \mathbb{Z}_+^n$ with $\sum_i p_i x_i \leqslant B$ that **maximises** $C_{\max}^{\mathrm{LPT}}(x)$ — the makespan when jobs are scheduled by LPT on m machines.

Two roles in the algorithm:

1. Solve once at start $\Rightarrow$ **strong initial incumbent**
2. Solve for remaining items at each node $\Rightarrow$ **tighter upper bound**

Tighter than ceiling bound because it respects the actual LPT structure instead of the relaxed $\lceil \cdot / m \rceil$ approximation.

Key result

Max-LPT can be solved **optimally** in $O(n \cdot B)$ via dynamic programming.

Approximation Guarantee — Two Lemmas

Lemma (Properties of α-approximations)

Let $\mathtt{approx}$ be an α-approximation for a minimisation problem, $\beta = 1/\alpha$. Then:

$$\beta \cdot \mathrm{Opt} \leqslant \beta \cdot \mathrm{approx} \leqslant \mathrm{Opt} \leqslant \mathrm{approx}$$

Theorem (Approximation & Bilevel [Weninger & Fukasawa 2025])

If $\mathrm{approx}(x)$ satisfies $\beta \cdot C_{\max}^{\mathrm{OPT}}(x) \leqslant \beta \cdot C_{\max}^{\mathrm{LPT}}(x) \leqslant C_{\max}^{\mathrm{OPT}}(x) \leqslant C_{\max}^{\mathrm{LPT}}(x)$ for all x, and $\hat{x} \in \arg\max_x C_{\max}^{\mathrm{LPT}}(x)$, then:

$$\beta \cdot \mathrm{Opt} \leqslant C_{\max}^{\mathrm{OPT}}(\hat{x}) \leqslant \mathrm{Opt}$$

Proof of second theorem: by optimality of $\hat{x}$ for Max-LPT, $C_{\max}^{\mathrm{LPT}}(x^*) \leqslant C_{\max}^{\mathrm{LPT}}(\hat{x})$. Chain together:

$$= \beta\, C_{\max}^{\mathrm{OPT}}(x^*) \leqslant \beta\, C_{\max}^{\mathrm{LPT}}(x^*) \leqslant \beta\, C_{\max}^{\mathrm{LPT}}(\hat{x}) \leqslant C_{\max}^{\mathrm{OPT}}(\hat{x}) \leqslant \mathrm{Opt}.$$

The $\frac{3}{4}$-Approximation Guarantee

Theorem ($\frac{3}{4}$-Approximation)

Let $\hat{x}$ be optimal for Max-LPT and x^* be optimal for the bilevel problem. Then:

$$\frac{3}{4} \cdot C_{max}^{OPT}(x^*) \leqslant C_{max}^{OPT}(\hat{x}) \leqslant C_{max}^{OPT}(x^*)$$

Proof:

- Graham (1969): LPT is a $\frac{4}{3}$-approximation for $P \,||\, C_{max}$, so $\beta = \frac{3}{4}$
- Lemma gives: $\frac{3}{4} C_{max}^{OPT}(x) \leqslant \frac{3}{4} C_{max}^{LPT}(x) \leqslant C_{max}^{OPT}(x) \leqslant C_{max}^{LPT}(x)$ for all x
- These satisfy conditions of the Bilevel Theorem with $\beta = \frac{3}{4}$
- Apply Bilevel Theorem $\Rightarrow$ result follows $\square$

Practical implication

Solving Max-LPT gives an incumbent **guaranteed** to be at most 25% worse. In practice the gap is nearly zero — as our experiments confirm.

Max-LPT Algorithm

Key observation: in an optimal Max-LPT solution, the last scheduled job I lands on the least-loaded machine. Makespan $= C_{max}^{(m)} + d_I$.

Algorithm MAX-LPT:

For each candidate last job $J \in \{1, \ldots, n\}$:

1. Set per-machine budget $B^* = \lfloor B/m \rfloor$
2. While $\hat{B} = B - mB^* < p_J$: reduce B^*
3. Lookup $DP(J, B^*)$: best duration-sum for types $1, \ldots, J$ with budget B^*
4. Candidate makespan: $\ldots J, B^*) + d_J$

Return $z^* = \max_J z$ — running

Why this is optimal

Property 1: B^* is the highest per-machine budget for the least-loaded machine

Property 2: $DP(J, B^*)$ gives the best load achievable on that machine

$\Rightarrow$ No combination can yield a larger $C_{max}^{(m)}$ for the available budget.

Early Optimality Detection

Key fact

For any x: $C_{max}^{LPT}(x) \geqslant C_{max}^{OPT}(x)$ (LPT only overestimates optimal makespan)

Let x^* be the Max-LPT optimal solution: $x^* \in \arg\max_x C_{max}^{LPT}(x)$.

If $C_{max}^{LPT}(x^*) = C_{max}^{OPT}(x^*)$ **then** for any feasible x:

$$\underbrace{C_{max}^{OPT}(x^*)}_{\text{our solution}} = C_{max}^{LPT}(x^*) \geqslant C_{max}^{LPT}(x) \geqslant C_{max}^{OPT}(x)$$

So x^* is globally optimal — **no branch-and-bound search needed!**

Practical significance

- $C_{max}^{OPT}(x^*)$ is computed anyway to initialise the incumbent (via Gurobi)
- Checking $C_{max}^{LPT}(x^*) = C_{max}^{OPT}(x^*)$ costs **nothing extra**
- In $\approx$ **84.4**% of tested instances $\Rightarrow$ solved at the root node alone

Full Branch-and-Bound Algorithm

B&B with early optimality detection:

1. Solve Max-LPT $\rightarrow$ get x_{LPT}
2. Compute $C_{max}^{LPT}(x_{LPT})$ and $C_{max}^{OPT}(x_{LPT})$
3. **If** $C_{max}^{LPT} = C_{max}^{OPT}$**: return** x_{LPT} $\leftarrow$ 84% of instances
4. $(x^*, C_{max}^{inc}) \leftarrow (x_{LPT}, C_{max}^{OPT}(x_{LPT}))$
5. **While** queue $Q \neq \emptyset$:
   - Pop node x at depth d
   - If complete: solve follower, update C_{max}^{inc}
   - Else: compute $UB(x_1, \ldots, x_d, B_{rem})$
   - If $UB \leqslant C_{max}^{inc}$: **prune**
   - Else: branch, add children to Q

Design choices

- **Depth-first search** for fast incumbent updates
- **Incremental LPT** maintains machine loads $O(m)$ per node
- **DP precomputation** makes every bound a $O(1)$ lookup
- **Leaf optimisation**: only evaluate the leaf that spends the most remaining budget

Experimental Setup & Correctness Verification

Environment:

- AMD Ryzen 5 9600X, 32 GB RAM
- Python 3.11; Gurobi 12.0.3; single-threaded

PatternTestSet: 140 instances (7 patterns)

Uniform Ratios, High Variance, Increasing Complexity, Random Moderate, Extreme, Strong Correlation, Subset Sum

6–12 job types, 3–8 machines, budget 15–50

Verification vs. complete enumeration

- 140 instances, **zero discrepancies** in optimal makespan
- Enumeration timed out on 15/140 (1 h limit)
- B&B solved all 140 in milliseconds

Speed-up

Up to **6 orders of magnitude** faster than complete enumeration

Grid Sensitivity: Status Distribution

Grid: 6×6 (job-types $\times$ machines), 3 budget multipliers, 5 seeds = 540 instances

Budget mult.	Root Optimal	Completed	Timeout	Leaf T/O
1.3	154 – 154	22 – 26	2 – 0	2 – 0
2.5	150 – 150	19 – 22	3 – 0	8 – 8
5.0	153 – 153	20 – 23	4 – 1	3 – 3

Format: Ceiling bound – MaxLPT bound

Key finding

84.6% of all instances solved at the root node by both bounds

MaxLPT advantage

More completed solves and fewer timeouts in every multiplier setting

Ceiling vs. MaxLPT: Hard Instances

Analysis of 61 non-root-optimal instances solved by both bounds:

Runtime (Ceiling − MaxLPT):

- Mean: **+71 s** (ceiling slower)
- Median: +0.4 s
- Max: **+1057 s**
- Ceiling faster: 20 times
- MaxLPT faster: **41 times**

Node count (Ceiling − MaxLPT):

- Mean: **+4.8M nodes**
- Median: +69K
- Max: +54.5M
- **MaxLPT always explores fewer nodes**

Interpretation

MaxLPT provides tighter pruning in every hard instance. The cost of precomputing n DP tables is fully recovered by the reduced node count.

Comparison with MibS Solver

MibS: state-of-the-art general-purpose bilevel MILP solver [Tahernejad et al. 2020]

Subset: 180 instances, $m \in \{4, 8, 12\}$, $j \in \{4, 8, 16, 20\}$, 3 multipliers, 5 seeds

$m \setminus j$	4	8	16	20
4	15/15	8/15	2/15	1/15
8	5/15	0/15	0/15	1/15
12	2/15	0/15	0/15	0/15

MibS successes out of 15. Our MaxLPT B&B solved **179/180**.

Speedup over MibS (1 h time limit)

Ceiling: mean +2877 s, median +3600 s MaxLPT: mean +2960 s, median +3600 s

Instances MibS cannot solve in 1 hour our algorithm solves in **milliseconds to seconds**.

Summary & Conclusions

Main contributions:

1. Formal model: bilevel knapsack-scheduling
2. Ceiling bound with DP precomputation
3. Max-LPT problem: solved in $O(n \cdot B)$
4. $\frac{3}{4}$-approximation guarantee
5. **Early optimality detection** — 84% solved at root
6. Comprehensive computational study

7. Transforms MibS by **orders of magnitude** on this problem class.

Limitations

- No theory for when root-optimality holds
- Exponential for > 30 job types without root-solve
- Parallel machines only

Future work

- Unrelated machines (ceiling bound promising)
- Stochastic processing times
- Characterise easy instances theoretically

Thank you!

Questions?

Ole Damke • o.damke@tu-braunschweig.de

Institut für Mathematische Optimierung • TU Braunschweig

References I

- R. L. Graham. Bounds on Multiprocessor Timing Anomalies. SIAM J. Appl. Math., 17(2):416–429, 1969.
- N. Weninger, R. Fukasawa. A fast combinatorial algorithm for the bilevel knapsack problem with interdiction constraints. Math. Program., 210(1):847–879, 2025.
- S. Tahernejad, T. K. Ralphs, S. T. DeNegre. A branch-and-cut algorithm for mixed integer bilevel linear optimisation problems. Math. Program. Comput., 12(4):529–568, 2020.
- T. Kleinert et al. A survey on mixed-integer programming techniques in bilevel optimisation. EURO J. Comput. Optim., 9:100007, 2021.
- S. Dempe, A. Zemkoho. Bilevel Optimization, vol. 161. Springer, 2020.